

Finer-Grained Subgraphs for KG-Commit

A Complete Survey of All Code Representation Options
with Efficiency, Delta Modeling, Implementation, Pros and Cons

KG-Commit Research Project

June 8, 2026

Legend for shaded analysis boxes:

- **Efficiency** — storage, maintenance, and query cost in the KG
- **Difference Modeling** — how commit-level deltas are extracted and stored
- **Prior Implementations** — readiness and difficulty of adoption
- **Pros** — advantages specific to KG-Commit
- **Cons** — disadvantages and risks specific to KG-Commit

Contents

1	Introduction	2
1.1	The Granularity Ladder	2
1.2	Key Principle	2
1.3	General Design Axes	2
2	Family 1: Token and Lexical Representations	3
2.1	What It Captures	3
3	Family 2: Syntactic Representations (AST)	4
3.1	What It Captures	4
4	Family 3: Control-Flow Representations	5
4.1	What It Captures	5
5	Family 4: Data-Flow Representations	6
5.1	What It Captures	6
6	Family 5: Unified Dependence Representations (PDG / SDG / CPG)	7
6.1	What It Captures	7
7	Family 6: Formal and Semantic Analysis	8
7.1	Abstract Interpretation	8
7.2	Alias and Points-To Analysis	9
7.3	Taint / Information-Flow Analysis	9
7.4	Type Systems and Effect Types	10

7.5	Symbolic Execution and Path Constraints	11
7.6	Formal Contracts (JML / Design by Contract)	11
8	Family 7: Program Slicing	12
8.1	What It Captures	12
9	Family 8: IR and Low-Level Representations	13
9.1	What It Captures	13
10	Family 9: Dynamic and Runtime Representations	14
10.1	What It Captures	14
11	Family 10: Change and Delta Representations	14
11.1	What It Captures	14
12	Family 11: Natural Language and Multi-Modal Representations	15
12.1	What It Captures	15
13	Family 12: Code Metrics and Feature Vectors	16
13.1	What It Captures	16
14	Family 13: Clone and Pattern Representations	17
14.1	What It Captures	17
15	Family 14: Learned and Neural Representations	18
15.1	What It Captures	18
16	Family 15: Inter-Procedural and Architectural Representations	19
16.1	What It Captures	19
17	Family 16: Binary and Compiled Representations	20
17.1	What It Captures	20
18	Universal Graph Encoding of Non-Graph Representations	21
19	Comprehensive Comparison	22
20	Recommended Phased Integration Plan	23
21	Open Research Questions	23

1 Introduction

KG-Commit models repository evolution as an incremental knowledge graph updated with every arriving commit. The current graph captures a structural/declarative layer (COMMIT, FILE, CLASS, FUNCTION, VARIABLE, AUTHOR, TIME, INTERVAL, BRANCH, ISSUE). This document surveys every known finer-grained representation that can be added as a subgraph below the FUNCTION node, covering sixteen families in full detail.

For each family, in addition to what it captures and how it is encoded as a graph, this version adds five analysis dimensions critical for the KG-Commit setting:

1. **Efficiency** — space, maintenance, and query cost
2. **Difference Modeling** — how commit-level deltas are extracted and added incrementally
3. **Prior Implementations and Difficulty** — tool readiness and implementation effort
4. **Pros** specific to KG-Commit
5. **Cons** specific to KG-Commit

1.1 The Granularity Ladder

```
FUNCTION
  BASIC_BLOCK / STATEMENT_BLOCK
    STATEMENT (assign, if, loop, return, throw, call, try/catch)
      EXPRESSION (binary, unary, cast, instanceof)
        OPERATION (+, ==, &&, [], new, .field)
          OPERAND (variable use, field read, array element)
            LITERAL / CONSTANT (0, "str", null, true)
```

1.2 Key Principle

Defect signal lives in *edges*, not nodes. Evaluate every representation primarily on which new **edge types** it contributes.

1.3 General Design Axes

Axis	Question	Why it matters
What is captured	Syntax, control, data, memory, behaviour?	Which bug classes are expressible
Static vs. dynamic	Source/IR analysis vs. run-time traces	Dynamic needs re-execution per commit
Symbolic learned	vs. Interpretable subgraph vs. dense vector	Both wanted simultaneously
Snapshot change	vs. Whole-function state vs. diff/delta	JIT signal is in the <i>change</i>
Intra- vs. inter-proc.	Inside one method vs. across calls	Many defects cross boundaries

Axis	Question	Why it matters
Language scope	Java-specific vs. language-agnostic	Corpus is Apache Java
Cost / explosion	Build time and graph size per commit	Full repo histories risk explosion

2 Family 1: Token and Lexical Representations

2.1 What It Captures

A lexer splits source into typed tokens: keywords, identifiers, literals, operators, separators. A function becomes a flat sequence $\langle t_1, t_2, \dots, t_n \rangle$. Variants include raw token sequences, type-only (values stripped), filtered/canonical (identifiers renamed to $v_1, v_2, \dots$), n -grams, BPE/subword tokenisation (CodeBERT-style), and grammar-aligned tokenisation.

Graph encoding.

```
Statement --TOKEN_SEQ[pos=i]--> Token(type, value)
Token(i) --NEXT_TOKEN--> Token(i+1)
```

Efficiency. Linear in function size: $O(n)$ nodes and $O(n)$ edges for n tokens. No complex structures. A 100-line function produces roughly 300–600 token nodes. Storage per commit is tiny (only changed functions’ tokens). Query cost for simple pattern matching is $O(n)$ traversal. The most space-efficient structural representation possible.

Difference Modeling. Excellent fit. The git diff already gives added/removed lines; tokenising each changed line is trivial. Delta = set of added and removed token nodes with their NEXT_TOKEN edges reconnected. No ambiguity: token positions are linear. Space for delta: proportional to changed tokens only. Time to extract: $O(k)$ where k = changed tokens. Fully deterministic no mapping problem.

Prior Implementations and Difficulty. Fully production-ready. tree-sitter, Java-Parser, ANTLR all produce token streams. Token-level diffs are trivially computable. DeepJIT (MSR 2019) and CC2Vec (ICSE 2020) use token sequences as their primary input. Integrating into KG-Commit requires one afternoon of engineering. Difficulty: **very easy**.

Pros. Cheapest representation possible. Works for all JVM languages in the Apache corpus (Java, Kotlin, Groovy, Scala). Perfect as a warm-start baseline before adding semantic layers. Token embeddings (BPE) are the input to all pretrained code models, so this layer is reused by Family 14.

Cons. Zero semantic content. Two programs with the same token bag can behave completely differently. Cannot detect data-flow, alias, or control-flow bugs. Sensitive to formatting changes (a whitespace refactor changes many tokens). BPE tokenisation misaligns with grammatical boundaries (TokDrift 2024), reducing the reliability of downstream model reasoning.

3 Family 2: Syntactic Representations (AST)

3.1 What It Captures

The Abstract Syntax Tree maps source code to a hierarchical tree of language constructs (IfStmt, ForStmt, MethodCallExpr, BinaryExpr, VarDeclarator) with leaves as identifiers and literals. Variants: CST (includes punctuation), normalised AST (identifiers renamed), AST paths (code2vec/code2seq leaf-to-leaf paths), type-resolved AST.

Graph encoding.

```
Function --AST_ENTRY--> RootNode
RootNode --AST_CHILD[pos=i]--> ChildNode
ChildNode --AST_CHILD[pos=j]--> LeafNode
LeafNode --HAS_TYPE--> TypeNode
```

Efficiency. $O(n)$ nodes for n source tokens, typically $2\text{--}5\times$ more nodes than tokens. A 100-line function produces 500–2000 AST nodes. Tree traversal is $O(n)$. Incremental parsing via tree-sitter reduces re-parse cost on small edits to $O(k \log n)$ for k changed characters. Memory footprint is moderate: one pointer per parent–child pair. Total AST storage for a full Apache project history is in the tens of millions of nodes — manageable with node compression (store type+value only, no source text duplication).

Difference Modeling. GumTree (ASE 2014, redesigned ICSE 2024) computes the AST edit script between before/after versions in $O(n^2)$ worst case but $O(n \log n)$ in practice for the 2024 redesign. The edit script is a compact list of **insert/delete/update/move** operations at AST-node granularity. Delta space: proportional to the number of changed AST nodes (typically $O(k)$ for k changed lines). Main challenge: GumTree edit scripts are *not unique* — different algorithms produce different scripts for the same semantic change, introducing ambiguity in graph updates. The 2024 redesign reduces this significantly.

Prior Implementations and Difficulty. Fully production-ready. GumTree is a Java library with CLI. JavaParser and Spoon produce ASTs directly. tree-sitter provides incremental, multi-language ASTs. GumTree integration with Java source is one-command. The Spoon library also provides type-resolved ASTs with no extra effort. Difficulty: **easy**.

Pros. Language-aware structural change detection far superior to line-based diff. Foundation for all higher-level representations — every other family builds on the AST. Type-resolved AST (Spoon) directly connects to your existing **DATA_TYPE** and **FUNCTION_SIGNATURE** nodes. AST paths (code2vec) are a cheap way to get function embeddings without a full LLM.

Cons. No execution semantics. Two ASTs can be structurally similar but behaviourally opposite (e.g., `>` vs. `>=`). Edit-script ambiguity makes deterministic graph updates non-trivial. Normalised ASTs lose identifier names, which are often semantically critical (`isNull()` vs. `isValid()`).

Java tooling. tree-sitter, JavaParser, Spoon, Eclipse JDT, GumTree.

4 Family 3: Control-Flow Representations

4.1 What It Captures

The Control Flow Graph (CFG) has nodes = basic blocks or statements, edges labelled TRUE/FALSE/UNCONDITIONAL. The Control Dependence Graph (CDG) captures “whether B executes depends on A.” The Call Graph (CG) connects callers to callees. The Exception Flow Graph models throw/catch/finally chains. Concurrency graphs (lock graphs, happens-before, race graphs) model threading.

Graph encoding.

```
Block --CFG_TRUE/FALSE/NEXT--> Block
Statement --CDG_CONTROLS--> Statement
Method --CALLS[site=s]--> Method
Method --THROWS--> ExceptionType
Block --ACQUIRES_LOCK--> LockNode
```

Efficiency. CFG: $O(n)$ nodes and $O(n)$ edges for n statements (branching factor typically 1–3). Basic-block CFG is 3–10× more compact than per-statement. Call graph: $O(M)$ nodes and $O(C)$ edges for M methods and C call sites — very compact (10^4 – 10^5 edges for a medium project). Lock graphs are sparse and tiny (few dozen nodes per application). CDG adds $O(n)$ extra edges to an existing CFG. Total: moderate overhead above AST, manageable with basic-block granularity.

Difference Modeling. Moderate difficulty. CFGs lack a natural diff operation — when a function changes, its CFG must be recomputed from scratch for that function. Practical approach: apply the GumTree AST edit script to identify which basic blocks were affected, then rebuild only those blocks. Call graph: update edges for new/removed/changed call sites only (incremental, cheap). Exception flow: update when **throws/catch** clauses change. Lock graph: update when **synchronized** blocks change. Main ambiguity: block identity across versions (a renamed block is the same or a new node?).

Prior Implementations and Difficulty. CFG construction: production-ready in Soot and Spoon. Call graph: Soot (CHA/RTA/SPARK) and WALA are production tools. CFG *differencing* (incremental CFG update) is much less mature — a custom implementation is required. Exception flow graph: research tools (Robusta, 2008) but not production-ready for incremental use. Concurrency graphs: RacerD (Meta) is production-ready but not designed for KG integration. Difficulty: CFG construction **easy**; incremental CFG diff **moderate**; concurrency graphs **hard**.

Pros. CFG gives cyclomatic complexity and loop detection for free. Call graph connects directly to your existing **FUNCTION** and **EXTERNAL_PACKAGE** nodes with no new node types. Exception flow graph is particularly valuable for Apache Java projects where exception mishandling is a major bug class. Lock graphs detect deadlocks — a real defect class in Apache concurrent components (ActiveMQ, etc.).

Cons. No data semantics — CFG alone cannot detect null-pointer dereferences or stale-value bugs. Incremental CFG update requires custom engineering. Context-insensitive call graph is imprecise for virtual dispatch in Java (produces spurious edges). Context-sensitive CG (SPARK/k-CFA) is expensive ($O(n^3)$ worst case).

Java tooling. Soot (CFG, CG), WALA (precise CG), Spoon (source-level CFG), RaceRD (race/lock analysis).

5 Family 4: Data-Flow Representations

5.1 What It Captures

The Data Flow Graph (DFG) connects each *definition* of a variable to every *use* it can reach (reaching definitions, def-use chains). SSA (Static Single Assignment) form makes each variable assigned exactly once, making def-use trivial and exact. Variants: use-def chains, live-variable analysis, available expressions, heap data flow.

Graph encoding.

```
Operand(def,var=v) --DDG_REACHES--> Operand(use,var=v)
Operand(use) --USE_DEF--> Operand(def)
Statement --KILLS_DEF--> Operand(def)
```

Efficiency. Moderate. For n statements and m variables, the number of def-use edges is $O(n \cdot m)$ worst case, but in practice $O(n)$ for local variables (most variables have few definitions and uses). SSA form (Soot Shimple) is more compact: each variable has exactly one def-node. However, *inter-procedural* data flow (tracking values across method boundaries) explodes combinatorially. Restricting to intra-procedural (per-function) data flow is the only practical choice for a full-history KG.

Difference Modeling. Hard. Def-use chains are *global within a function*: changing one assignment can invalidate many reaching-definition edges. A single added statement may kill existing def-use chains and create new ones throughout the function. Incremental data-flow analysis algorithms exist (Ryder 1988, Alpern et al. 1990) but are complex to implement and rarely used in practice. The most realistic approach: recompute the full DFG of any changed function. Store only the new DFG and diff it against the old at the edge level. This is correct but requires full re-analysis of each changed function per commit.

Prior Implementations and Difficulty. Soot Jimple provides def-use chains over a 3-address IR. GraphCodeBERT’s data flow extractor (Python-only in the original paper) has been ported to Java by several groups. WALA provides inter-procedural def-use. All are production-ready for *batch* computation. *Incremental* def-use update: not available off-the-shelf — requires a custom implementation using Soot as the underlying analysis engine. Difficulty: batch **easy**; incremental **hard**.

Pros. The single highest-yield semantic relation for defect prediction. GraphCodeBERT’s choice of data flow as its sole structural signal is strong empirical evidence. Directly encodes null-pointer, use-before-init, and stale-cache bugs — the most common Apache defect classes. Clean graph structure with a small vocabulary.

Cons. Imprecise for OO Java without alias analysis (field writes through references are not captured by local def-use). Incremental update requires full re-computation of changed functions. Requires IR compilation (Soot Jimple), adding a build-system dependency.

Java tooling. Soot (Jimple def-use, Shimple SSA), WALA, Spoon (source-level approximations).

6 Family 5: Unified Dependence Representations (PDG / SDG / CPG)

6.1 What It Captures

The Program Dependence Graph (PDG) merges control dependence (CDG) and data dependence (DDG). The System Dependence Graph (SDG) is the inter-procedural PDG. The Code Property Graph (CPG) further unifies $AST \cup CFG \cup PDG$ into one multigraph, queryable with a graph traversal DSL.

Graph encoding.

```
Function --HAS_CPG_ENTRY--> EntryNode
EntryNode --AST_CHILD[pos]--> ChildNode
Block --CFG_TRUE/NEXT--> Block
Predicate --CDG_CONTROLS--> Statement
Operand(def)--DDG_REACHES--> Operand(use)
CallExpr --CALLS--> Function
CallExpr --ARG_TO[pos]--> Param
```

Efficiency. High cost. The CPG = AST + CFG + PDG stacks three graphs on the same node set, multiplying edge count by 3–10× relative to AST alone. A medium function (50 lines) produces 500–2000 CPG nodes and 1000–5000 CPG edges. For a project with 10^5 functions and full commit history, the total CPG could reach 10^8 – 10^9 edges. Joern mitigates this by storing CPGs in a specialised graph database (ShiftLeft Ocular / Neo4j backend). Basic-block granularity (rather than per-statement) reduces node count by 3–8×.

Difference Modeling. Very hard. All three CPG layers are coupled: a changed statement touches AST nodes, CFG edges, and potentially many PDG edges simultaneously. No incremental CPG update algorithm is currently available. The practical approach (used in GraphSPD, S&P 2023) is to: (1) build the CPG of the changed function *before* the commit, (2) build it *after*, (3) union them, linking corresponding nodes with MAPPED_TO edges and tagging changed nodes with DELTA_STATUS. This doubles the CPG size for changed functions but is semantically clean and the only unambiguous delta representation.

Prior Implementations and Difficulty. Joern (open-source, Java/Scala) is production-ready and generates CPGs from Java source or bytecode with a single command. It ships with a Gremlin/DSL query interface. The delta-CPG (before+after union) is *not* built into Joern — a wrapper script must build both versions, emit two CPG files, and merge them. This is a moderate engineering task ($\approx$ 2–3 weeks). Difficulty: CPG construction **easy** (Joern); delta-CPG **moderate**.

Pros. Subsumes Families 1–4 in one coherent schema. The academically-proven backbone for vulnerability and defect detection (Devign, ReVeal, GraphSPD, dozens of 2024–2025 papers). Node/edge vocabulary maps directly onto your existing KG. Joern’s query DSL enables arbitrary pattern queries on the same graph. The single best

structural choice for the backbone subgraph.

Cons. Highest storage and computation cost of any representation. Joern has significant JVM startup overhead per analysis run. Delta-CPG construction requires two full CPG builds per changed function. The graph size can exceed KG storage budgets for full project histories at commit granularity. Not truly incremental.

Java tooling. **Joern** (recommended), Soot (custom SDG), WALA (precise inter-procedural SDG).

7 Family 6: Formal and Semantic Analysis

7.1 Abstract Interpretation

Over-approximates all possible program states by propagating abstract values through the CFG until a fixpoint. Domains: sign/nullness, intervals, octagons, polyhedra, tpestate, taint, constant propagation.

Graph encoding.

```
Variable --MAY_BE_NULL--> Statement(dereference)
Statement --PROPAGATES_TAINT--> Statement
Block --ABSTRACT_VALUE[domain=nullness]--> LatticeElement
```

Efficiency. Highly domain-dependent. Nullness analysis: $O(n)$ per function, very cheap. Interval analysis: $O(n^2)$ with widening. Polyhedra: PSPACE-hard, not usable per commit. Taint analysis with fixed source/sink sets: $O(n)$. For KG-Commit the practical choice is a restricted set of cheap domains: nullness + taint. These add a small number of typed edges (typically $O(n)$ per function) and are efficiently storable.

Difference Modeling. Hard in general. Abstract interpretation is a whole-function fixpoint computation — a single changed statement can propagate to many abstract states throughout the function. Infer (Facebook) implements an *incremental mode* (`-reactive`) that re-analyses only changed procedures, re-using cached summaries for unchanged callees. This is the only production-ready incremental abstract interpreter. For KG-Commit: run Infer in reactive mode per commit, diff the reported bugs (added/removed warnings) and store those as typed edges.

Prior Implementations and Difficulty. **Infer** (Meta, open-source) is production-ready for Java, runs on CI, and has incremental mode. SpotBugs/FindBugs provide pattern-based analysis (not true abstract interpretation but cheap and ready). Full abstract domains (polyhedra, octagons): ASTRÉE, Julia — research/commercial tools, not easy to integrate. Difficulty: Infer + reactive mode **easy** to **moderate**; full abstract domains **hard**.

Pros. Infer’s incremental mode is essentially designed for the commit-level analysis loop. It directly outputs “new bug introduced / existing bug removed” per commit, which maps cleanly to KG edge insertions/deletions. Nullness analysis is the single most relevant domain for Apache Java (NPE is the #1 bug class).

Cons. Infer has significant false-positive rates (especially for concurrent code). Full abstract interpretation is too expensive for rich domains. Incremental mode requires

a build system integration (Gradle/Maven) which adds pipeline complexity. Lattice-valued node attributes are not directly compatible with standard GNN aggregation.

Java tooling. Infer (Meta), SpotBugs, Julia, Soot (extensible dataflow framework).

7.2 Alias and Points-To Analysis

Answers “can pointers **a** and **b** reference the same heap object?” Produces a points-to graph, shape/heap graph, and alias sets.

Graph encoding.

```
Variable --POINTS_TO--> AbstractLocation
AbstractLocation --FIELD[f]--> AbstractLocation
Variable --MAY_ALIAS--> Variable
```

Efficiency. High cost. Andersen’s algorithm: $O(n^3)$ time and $O(n^2)$ space. Flow-insensitive Steensgaard: $O(n\alpha(n))$ but less precise. For large Apache projects, whole-program alias analysis produces millions of may-alias pairs. Practical mitigation: restrict to intra-procedural alias within changed functions only; use demand-driven alias (compute only for variables accessed in the changed code).

Difference Modeling. Extremely hard. Alias analysis is a global analysis — a single field assignment in one method can change the points-to graph for the entire program. No practical incremental alias analysis is available for Java. The only realistic approach: recompute alias for changed functions only (intra-procedural, demand-driven), accepting imprecision for inter-procedural cases.

Prior Implementations and Difficulty. Soot/SPARK and WALA are production tools for Java. Demand-driven alias (WALA Demand CG) is available. Incremental alias: not production-ready. Difficulty: batch intra-procedural **moderate**; full inter-procedural incremental **very hard**.

Pros. Enables sound data-flow analysis for OO Java (without alias, DFG is imprecise for heap-traversing code). MAY_ALIAS edges are a unique structural signal not present in any other family. Essential for field-access null-dereference bugs.

Cons. Prohibitively expensive for whole-project incremental analysis. Generates enormous numbers of may-alias pairs (most spurious). Not incrementally maintainable. Should be treated as a selective enrichment, not a default layer.

7.3 Taint / Information-Flow Analysis

Tags values at sources (user input, network) as tainted and tracks propagation to sinks (SQL queries, file writes, HTTP responses).

Graph encoding.

```
Operand --TAINT_SOURCE--> TaintLabel
Operand --TAINT_FLOWS_TO--> Operand
Operand --TAINT_SINK--> SinkType(SQL|HTTP|FILE)
Statement --SANITIZES--> TaintLabel
```

Efficiency. Linear in program size for fixed source/sink sets: $O(n)$ per function. Taint paths are sparse (few sources/sinks per function). Total taint edges across a project are modest. Queries (“is there a taint path from source s to sink k ?”) are simple reachability queries on the taint subgraph.

Difference Modeling. Moderate. Taint paths are explicit source→sink chains. When code changes, re-run taint analysis on changed functions. New taint paths introduced by the commit are new edges; broken paths are removed edges. Joern’s taint query system can be re-run incrementally by querying only changed CPG nodes.

Prior Implementations and Difficulty. FlowDroid (TU Darmstadt, open-source) for Android/Java. Joern taint queries for server-side Java. Checker Framework `@Tainted` type qualifier. All are production-ready. Integration into KG-Commit requires a source/sink configuration file. Difficulty: **moderate**.

Pros. Directly encodes injection and leakage bug classes. Apache web frameworks (e.g., Struts, CXF) have well-defined taint sources/sinks. Taint paths are interpretable, explicit graph edges.

Cons. Requires alias analysis for precision (otherwise false positives). Source/sink configuration is application-specific and must be maintained. Not useful for non-security defect classes.

7.4 Type Systems and Effect Types

Nullness types (`@NonNull/@Nullable`), ownership types, effect types (reads/writes/throws), typestate, gradual typing.

Graph encoding.

```
Variable --HAS_NULLNESS--> NullnessLabel(nullable|nonnull)
Method --HAS_EFFECT[kind]--> SideEffect
Variable --TYPESTATE[s]--> TypestateNode
```

Efficiency. Very cheap for annotation-based types (read annotations directly from source). Inferred type checking (NullAway, Checker Framework): $O(n)$ per function. Storage: one typed label per variable per program point — compact.

Difference Modeling. Simple. When a function changes, re-run the type checker on that function. New violations = new edges; resolved violations = removed edges. If annotations are added/removed in the commit, those are direct KG edge insertions/deletions.

Prior Implementations and Difficulty. NullAway (Uber, open-source, annotation-based nullness), Checker Framework (extensible pluggable type systems), JSpecify (emerging standard). All production-ready. Difficulty: **easy**.

Pros. Nullness types are the most directly NPE-predictive signal. Many Apache projects already use `@NonNull/@Nullable` annotations — these are free labels. Cheap to compute and store.

Cons. Annotation coverage is incomplete in most codebases. Inferred types are conservative (many unknowns). Not useful for logic bugs unrelated to typing.

Java tooling. NullAway, Checker Framework, JSpecify, Infer.

7.5 Symbolic Execution and Path Constraints

Replaces concrete inputs with symbolic variables, explores all execution paths, accumulates path constraints (conjunctions of predicates), checks feasibility with an SMT solver.

Graph encoding.

```
Block --PATH_CONDITION--> Formula(SMT_string)
Block --INFEASIBLE_PATH--> Block (UNSAT)
SymbolicState --FORKS_AT--> Predicate
```

Efficiency. Very high cost. The symbolic execution tree is exponential in the number of branches. For a function with 20 branches, the tree has up to 2^{20} leaves. Practical tools use heuristics (path merging, loop summarisation) to bound the tree. For a commit-level KG over a full project history, full symbolic execution is impractical. Selective use (changed functions only, bounded depth) is the only viable approach.

Difference Modeling. Hard. Symbolic execution trees are not differentiable. When code changes, the tree must be recomputed. However, *infeasibility edges* (“this CFG path is unreachable”) are a compact derived representation that can be stored as a small set of edges per function and re-derived cheaply for changed functions.

Prior Implementations and Difficulty. SPF (Symbolic PathFinder, JPF-based, open-source), JDart (Java), EvoSuite (concolic, production-ready for test generation). None are designed for incremental per-commit KG integration. Neuro-symbolic hybrid (NeuroSE, NDSS 2019) is research-grade. Difficulty: **very hard**.

Pros. Highest-precision analysis available. Infeasibility annotations directly prune false-positive paths. Path constraints can be stored as node properties and used as rich features for GNN message passing.

Cons. Exponential cost makes it impractical for routine per-commit analysis. Path explosion is not solvable in general. SMT solver calls add significant latency. Should be considered only for selective, high-value targets (e.g., security-critical functions flagged by taint analysis).

7.6 Formal Contracts (JML / Design by Contract)

Pre-conditions, post-conditions, class invariants as first-class program entities (JML annotations, `assert`, Guava Preconditions).

Graph encoding.

```
Method --PRECONDITION--> Formula
Method --POSTCONDITION--> Formula
Class --INVARIANT--> Formula
```

Efficiency. Very cheap when mined from existing annotations (`@requires`, `Objects.requireNonNull`, Guava checks). One node per contract per method. Storage is $O(M)$ for M annotated methods.

Difference Modeling. Simple. Annotation changes are directly detectable in the AST diff (a `@NonNull` annotation added or removed is a GumTree `insert/delete` action). New/removed contract nodes are straightforward KG updates.

Prior Implementations and Difficulty. OpenJML (JML checker, open-source), ESC/Java2 (research), Checker Framework (annotation-based contracts). Mining implicit contracts from `assert` statements: trivial via AST traversal. Difficulty: **easy** for annotation-based; **hard** for inferred contracts.

Pros. Cheap and interpretable. Apache projects use Guava Preconditions extensively. Contract violations are direct defect signals. Contract additions in a commit are a strong signal of a previous latent bug.

Cons. Most Apache code is not annotated with JML. Mining implicit contracts (from asserts) misses many real preconditions. Formal verification is expensive.

Java tooling. OpenJML, Checker Framework, Guava/Apache Commons Preconditions mining via AST.

8 Family 7: Program Slicing

8.1 What It Captures

A backward static slice with criterion $(stmt, var)$ = all statements that *may* have influenced *var* at *stmt* (PDG backward reachability). Forward slice = all statements *var* at *stmt* may influence. Dynamic slices restrict to a specific execution trace.

Graph encoding.

```
Statement --IN_BACKWARD_SLICE[criterion=(s,v)]--> SliceCriterion
Statement --IN_FORWARD_SLICE[criterion=(s,v)]--> SliceCriterion
SliceCriterion --OBSERVED_AT--> Statement
```

Efficiency. Static slice computation: $O(|PDG|)$ per criterion (graph traversal). If the PDG already exists, slices are almost free. Storing all slices for all criteria is expensive ($O(|PDG|^2)$ in the worst case). Practical approach: store only *slice sizes* as node attributes on Statement nodes (a highly predictive hand-crafted feature) rather than explicit slice membership edges. Dynamic slices: require trace execution, storage is proportional to trace length — expensive.

Difference Modeling. Moderate. Slices are derivable from the PDG, so if the PDG is incrementally updated (per changed function), slice criteria that overlap the change can be re-derived. Static backward slice size is a cheap commit-level feature: “how much code does the changed statement’s backward slice cover before vs. after the commit?”

Prior Implementations and Difficulty. **JavaSlicer** (TU Munich, dynamic slicing, open-source). Soot (static backward/forward slice via PDG). Both are production-ready. Integrated slicing for commit-level prediction is not off-the-shelf but is straightforward given Soot’s PDG. Difficulty: **moderate**.

Pros. Backward slice size of the changed statement is one of the most powerful single features for fault localisation. Directly answers “how much context does this change touch?” Derivable from the PDG — if you build the CPG, slices are a free by-product.

Cons. Dynamic slicing requires running tests per commit. Static slices are over-approximations (include infeasible paths). Storing full slice membership as edges is too

expensive. Useful mainly as scalar features derived from the PDG.

Java tooling. JavaSlicer (dynamic), Soot PDG (static), WALA.

9 Family 8: IR and Low-Level Representations

9.1 What It Captures

Java bytecode, Soot’s Jimple (typed 3-address IR), Shimple (SSA Jimple), ProGraML (language-agnostic IR graph combining control+data+call), IR2Vec (learned IR instruction embeddings).

Graph encoding. Same CPG vocabulary but over IR instructions rather than source statements. Adds `PHI_FUNCTION` nodes (SSA) and `LLVM_USE` / `IR_DEF` edges.

Efficiency. IR is 3–5× more verbose than source (many 3-address instructions per source line). Jimple for a 100-line Java function produces 400–800 instructions. SSA adds ϕ -functions at join points. Total storage is moderate per function. However, source→IR correspondence must be maintained (for alignment with diffs), adding book-keeping overhead.

Difference Modeling. Hard. IR instructions do not correspond 1:1 to source lines. Compiler transformations (inlining, constant folding, instruction scheduling) mean that a single source change can produce a large IR diff. Tracking the mapping from source diff to IR diff requires preserving `LineNumberTable` debug metadata. No standard tool for IR-level differencing exists.

Prior Implementations and Difficulty. Soot Jimple/Shimple: production-ready (used in hundreds of papers). Soot’s Java frontend maps source line numbers to Jimple instructions. ProGraML: designed for LLVM IR (C/C++) — a Java port requires Soot Jimple emission + custom ProGraML-style graph builder. IR2Vec: LLVM-native. Difficulty: Jimple extraction **easy**; ProGraML-style Java graph **moderate**; IR differencing **hard**.

Pros. More precise def-use analysis than source-level (no parsing ambiguities). Language-agnostic across all JVM languages. SSA form makes GNN message passing over def-use trivial. ProGraML’s design is directly applicable if using LLVM-compiled Java (via JVM bytecode→LLVM translators).

Cons. Mapping back to source locations (needed for diff alignment) is error-prone. Adds a Soot compilation dependency to the pipeline. IR representations are harder to interpret and debug. Not a natural fit for commit-level delta computation.

Java tooling. Soot (Jimple, Shimple), WALA (SSA-based IR), ASM (bytecode), ProGraML (LLVM IR).

10 Family 9: Dynamic and Runtime Representations

10.1 What It Captures

Execution traces (instruction/method-call sequences), runtime value traces, coverage matrices (test $\times$ statement binary matrix), dynamic invariants (Daikon), concolic execution.

Graph encoding.

```
TraceEvent --NEXT_EVENT[step=i]--> TraceEvent
TraceEvent --AT_STATEMENT--> Statement
Test --COVERS--> Statement
Test --RESULT--> PassFail
Method --DYNAMIC_PRECOND--> InvariantFormula
```

Efficiency. Very expensive. Full instruction traces for a Java application can be gigabytes per execution. Method-call traces are 100 $\times$ smaller but still large. Coverage matrices are compact ($|tests| \times |statements|$ binary matrix, typically sparse). Daikon invariants are $O(|variables|)$ per program point — compact. The real cost is *re-executing the test suite for each historical commit*, not storage.

Difference Modeling. Not applicable in the traditional sense. Traces are per-execution, not per-commit. To get commit-level dynamic features: check out each commit, compile, run tests, record coverage/traces. This pipeline is used by mutation testing research but is extremely expensive for full repo histories. Coverage diff (which statements gained/lost coverage between commits) is a useful lightweight variant.

Prior Implementations and Difficulty. **Daikon** (dynamic invariants, production-ready). **JaCoCo** (coverage, production-ready, integrates with Maven/Gradle). **JavaSlicer** (dynamic slicing, production-ready). **PIT** (mutation testing, production-ready). All require a working build and test suite. Historical commit re-execution: requires a CI-like pipeline per commit. Difficulty: per-commit re-execution infrastructure **hard**; using existing CI traces **moderate**.

Pros. Ground-truth runtime behaviour — eliminates static analysis false positives on the executed path. Coverage matrices are proven effective for fault localisation (ICSE 2021). Daikon invariants are highly interpretable and often catch pre/post condition violations that static analysis misses.

Cons. Requires compiling and running every historical commit — many commits in the Apache corpus do not compile cleanly in isolation. Test coverage is limited (uncovered paths produce no signal). Serious leakage risk: traces from post-fix commits include the fix; must use pre-commit test suite only.

Java tooling. Daikon, JaCoCo, JavaSlicer, PIT, EvoSuite.

11 Family 10: Change and Delta Representations

11.1 What It Captures

Line-based diffs, AST edit scripts (GumTree), delta CPG (before+after CPG union), semantic change taxonomies, hunk-level representations, bi-modal change+message embeddings (CC2Vec, BiCC-BERT).

Graph encoding.

```
Commit --EDITS[action=insert]--> ASTNode(after)
Commit --EDITS[action=delete]--> ASTNode(before)
ASTNode(bef)--MAPPED_TO--> ASTNode(aft)
ASTNode --CHANGED_IN--> Commit
CPGNode --STATUS--> {added|deleted|modified|unchanged}
```

Efficiency. Excellent. AST edit scripts are small: typically $O(k)$ operations for a commit touching k AST nodes. Delta CPG stores only changed-function subgraphs (before+after). Line-level diff: a handful of integers per changed line. This is by far the most storage-efficient semantic representation for a commit-centric KG. The delta *is* the representation — no redundant snapshot needed.

Difference Modeling. By definition, this family IS the difference. GumTree produces an edit script in $O(n \log n)$ for the 2024 redesign. The delta is unambiguous in the sense that it is computed from well-defined before/after snapshots. The only ambiguity is in edit-script minimality (multiple minimum-cost scripts may exist). Space: $O(k)$ for k changed nodes — optimal. The delta CPG (GraphSPD approach) stores before+after CPG for changed functions only, linking them with MAPPED_TO edges.

Prior Implementations and Difficulty. **GumTree:** production-ready Java library. **ChangeDistiller:** Java, research-grade. **CC2Vec:** PyTorch, research code available. **BiCC-BERT:** paper available, training code partially released. **Delta CPG** (before+after union): not a standalone tool — requires Joern + a merge script. GraphSPD (S&P 2023) provides a reference implementation for C/C++; a Java port requires moderate effort. Difficulty: GumTree **very easy**; delta CPG **moderate**; full BiCC-BERT pretraining **hard**.

Pros. Perfect fit for the JIT setting: the commit IS the delta. Minimal storage overhead. GumTree edit scripts are the most semantically precise change representation available. Directly anchors all other semantic layers (changed CPG nodes are the subset of the full CPG that need re-analysis). The open research gap (semantic change-graph for JIT-DP) is directly addressed here.

Cons. Delta CPG requires full CPG builds for before+after versions of changed functions, which is expensive. GumTree edit scripts are non-unique (multiple valid scripts for the same change). Inter-procedural change effects (commit edits method A, defect surfaces in caller B) are not captured by function-local deltas.

Java tooling. GumTree, ChangeDistiller, Joern (for CPG), CC2Vec (PyTorch), BiCC-BERT.

12 Family 11: Natural Language and Multi-Modal Representations

12.1 What It Captures

Commit messages, code comments (Javadoc, inline), bug reports and issue trackers (titles, descriptions, severity, stack traces), code review comments, API documentation and behavioural specifications.

Graph encoding.

```
Commit --HAS_MESSAGE--> MessageNode
MessageNode --HAS_INTENT--> IntentLabel(fix|feat|refactor)
Commit --FIXES--> Issue
Issue --CONTAINS_STACKTRACE--> StackTrace
StackTrace --FRAME[depth=i]--> StackFrame
StackFrame --AT_METHOD--> Function
ReviewComment --ON_LINE--> DiffLine
```

Efficiency. Very cheap. Commit messages are short strings (median ~ 8 words in Apache repos). Embedding vectors are fixed-size (768 or 1024 floats). Issue descriptions are larger but rare relative to commits. Stack trace frames are small (~ 10 – 50 frames). Total overhead: a few kilobytes per commit for text plus one embedding vector per entity. The cheapest semantic representation.

Difference Modeling. Natural — commit messages and review comments are inherently per-commit. Each new commit brings a new message node and (possibly) new issue/review links. No diff computation is needed: just append the new NL nodes and their edges. Issue links (FIXES #1234) are extractable from the commit message with a regex. Stack traces in issue descriptions can be parsed and linked to FUNCTION nodes. This is the cleanest “one node per commit” update model of any family.

Prior Implementations and Difficulty. `git log` provides commit messages. GitHub/Jira REST APIs provide issue data. BERT/roBERTa/CodeBERT embeddings via HuggingFace: one-line inference. BiCC-BERT (bi-modal change+message): pretraining code partially available. Intent classification (bug-fix vs. feature vs. refactor): SZZ-based labels, fine-tuned BERT classifiers exist. Stack trace parsing: custom regex, trivial. Difficulty: **very easy**.

Pros. Cheapest feature with high signal. Intent classification (“fix” commits are labelled defect-introducing in many studies). Stack traces directly link NL context to code locations. Review comments are oracle-quality labels (a reviewer flagging an NPE is nearly ground truth). Directly extends your existing COMMIT and ISSUE nodes — no new infrastructure.

Cons. Inconsistent quality: many Apache commits have poor or empty messages (“minor fix”, “WIP”). Issue linkage is incomplete (not all commits reference issues). Leakage risk: review comments appear *after* the commit is submitted; they may encode post-fix context.

Java tooling. `git log`, JGit, GitHub/Jira REST API, HuggingFace Transformers, custom stack-trace parser.

13 Family 12: Code Metrics and Feature Vectors

13.1 What It Captures

Scalar hand-crafted features: code complexity metrics (cyclomatic complexity, LOC, nesting depth, fan-in/out, Halstead volume), OO metrics (CK suite: WMC, DIT, NOC, CBO, LCOM, RFC), process/change metrics (churn, age, # authors, fix frequency, change burst).

Graph encoding.

```
Function --HAS_METRIC[name=cyclomatic]--> MetricValue(float)
Class --HAS_METRIC[name=CB0]--> MetricValue(float)
File --HAS_METRIC[name=churn_30d]--> MetricValue(float)
```

Efficiency. Extremely efficient. A handful of scalar values per function/class. Storage: $O(F \times M)$ for F functions and M metrics — a tiny fraction of any other representation. Queries are $O(1)$ lookups. No complex graph traversal needed. This is the cheapest possible semantic enrichment.

Difference Modeling. Trivial. Re-compute metrics only for changed functions/-classes per commit. Store the *delta* metric value (new – old) as an additional edge. “Cyclomatic complexity increased by 3 in this commit” is a directly stored property. Process metrics (churn, age, # authors) are updated with each commit by simple counter increments. Completely unambiguous.

Prior Implementations and Difficulty. **CK** (Java CK metrics library, one-JAR, production-ready). **PMD**, **Checkstyle**, **SonarQube** (production-ready, CI-integrated). **JavaNCSS** for NCSS/Halstead. Git-derived process metrics: trivially computable from `git log -numstat`. Your existing `graph.py` already computes `la` and `ld` — this is an extension of that pattern. Difficulty: **very easy**.

Pros. Proven baselines in defect prediction literature. Directly plug into your existing aggregate-feature design in `graph.py` with zero GNN engineering. Interpretable and explainable. Process metrics (fix frequency, author count) are among the strongest predictors in JIT-DP studies (LaPredict, JITLine).

Cons. Surface-level proxies. Many metrics (LOC, complexity) plateau in predictive power with more training data. Cannot encode semantic bugs (a perfectly simple function can still be buggy). Metrics ignore what the code *does*.

Java tooling. CK, PMD, SonarQube, JavaNCSS, git log.

14 Family 13: Clone and Pattern Representations

14.1 What It Captures

Code clone pairs (Type 1–4), design pattern instances (Singleton, Observer, Factory...), code smells (God Class, Long Method, Feature Envy, Data Clumps), Self-Admitted Technical Debt (SATD from TODO/FIXME comments).

Graph encoding.

```
Function --IS_CLONE_OF[type=2]--> Function
Class --IMPLEMENTS_PATTERN--> DesignPattern(name)
Class --HAS_SMELL--> CodeSmell(name=GodClass)
Comment --IS_SATD--> SATDCategory(design|defect)
```

Efficiency. Clone detection: $O(n^2)$ naive pairwise, but LSH/shingling-based tools (SourcererCC) scale near-linearly. Clone graph is sparse in practice. Smell detection: $O(n)$ once metrics are computed (derived from CK metrics). SATD detection: $O(n)$ comment scanning. Design pattern detection: $O(n)$ AST structural matching. All are

cheap relative to semantic analyses.

Difference Modeling. Clone detection: new clones can be introduced (copy-paste of a changed function) or existing clones broken (one copy diverges). Incremental clone detection requires re-checking only the changed functions against all existing functions. SourcererCC supports an incremental mode. Smell detection: re-run only on changed entities per commit ($O(1)$ per changed function). SATD: directly detectable in the diff (added/removed TODO/FIXME comments).

Prior Implementations and Difficulty. SourcererCC (industrial-strength clone detection, production-ready, incremental mode). iClones (incremental, research-grade). Code smell detection: **PMD** rules, **SonarQube**, ML-based (ML-smell 2025). Design pattern detection: research tools (PatternBox, DPD). SATD: **SATDDetector** (open-source, uses SVM on comment text). Difficulty: smells + SATD **easy**; clone detection **moderate**; design patterns **moderate**.

Pros. Clone pairs create explicit cross-file edges — a bug fixed in one clone but not the other is directly detectable. SATD (TODO/FIXME) is one of the cheapest and most proven defect predictors. Code smells are interpretable proxies for architectural degradation. Design pattern violations can signal misuse.

Cons. Clone detection at scale (10^5 functions $\times$ full history) is expensive. Design pattern detection has false positives. Smells are noisy proxies — many God Classes have no bugs. SATD detection accuracy depends on comment quality.

Java tooling. SourcererCC, PMD, SonarQube, SATDDetector.

15 Family 14: Learned and Neural Representations

15.1 What It Captures

Pre-trained code LMs (CodeBERT, GraphCodeBERT, CodeT5, UniXcoder), GNN architectures (GCN, GAT, GGNN, R-GCN, HGT), path-based embeddings (code2vec, code2seq, ASTNN), contrastive/self-supervised code representations (DOBF, DISCO).

Graph encoding. Node embeddings as attributes:

```
Node --HAS_EMBEDDING[model=GraphCodeBERT]--> Vector(float[768])
```

GNN propagation over the CPG / delta-CPG structure.

Efficiency. Inference cost: a CodeBERT forward pass takes $\sim 50\text{--}200\text{ms}$ per function on GPU, $\sim 2\text{--}10\text{s}$ on CPU. Storing embedding vectors: $768 \times 4 \text{ bytes} = 3\text{KB}$ per node. For 10^6 function nodes: $\sim 3\text{GB}$ just for embeddings. GNN re-propagation per changed subgraph: $O(|edges| \times \text{layers})$. GPU infrastructure is required for production-speed embedding generation. Total cost is high but *amortisable*: compute once and store; recompute only for changed functions.

Difference Modeling. When a function changes, its token sequence changes $\Rightarrow$ its LM embedding must be recomputed (one forward pass per changed function, $\sim 100\text{ms}$ on GPU). The GNN propagation must also be re-run for the changed subgraph (neigh-

bourhood of changed nodes). For a commit touching 10 functions: $\sim 1\text{--}2\text{s}$ on GPU. The incremental update is clean: identify changed function nodes, recompute their embeddings, re-run GNN propagation on their neighbourhoods.

Prior Implementations and Difficulty. **HuggingFace Transformers:** all major code LMs are one-line inference. **PyTorch Geometric / DGL:** production-ready GNN libraries. Devign (GGNN over CPG): reference code available (research-grade). End-to-end JIT-DP with GNN: no production-ready pipeline exists, but the components are all available. Difficulty: embedding generation **easy**; GNN training on CPG **moderate**; full end-to-end JIT-GNN **hard**.

Pros. State-of-the-art accuracy on most code intelligence benchmarks. Pre-trained models transfer across projects with no fine-tuning. Captures semantic meaning of identifier names (“isNull” vs. “validate”). GraphCodeBERT’s data-flow-aware pre-training aligns exactly with Family 4. HGT handles your heterogeneous node/edge types natively.

Cons. Black-box: not interpretable. Requires GPU infrastructure. Embedding recomputation per changed function adds latency to the online learning loop. Pre-trained models may be biased toward common programming patterns (underrepresented in Apache-style server-side Java). Very high engineering complexity to integrate end-to-end.

Java tooling. HuggingFace Transformers, PyTorch Geometric, DGL, Joern + custom GNN wrapper.

16 Family 15: Inter-Procedural and Architectural Representations

16.1 What It Captures

Context-sensitive call graphs (k-CFA, object-sensitive), module/package dependency graphs, co-change graphs (files that change together in commits), developer expertise and ownership graphs.

Graph encoding.

```
Package --DEPENDS_ON--> Package
Package --VIOLATES_LAYER--> Package
File --CO_CHANGES_WITH[freq=n]--> File
Author --OWNS[fraction=0.7]--> File
```

Efficiency. Call graph: $O(M+C)$ for M methods and C call sites — compact. Module dependency: $O(P^2)$ worst case for P packages but very sparse in practice ($P \approx 100\text{--}1000$). Co-change graph: $O(F^2)$ worst case for F files but sparse ($F \approx 10^3\text{--}10^4$). Ownership vectors: $O(F \times A)$ for A authors — compact. All are low-storage relative to intra-procedural analyses.

Difference Modeling. Naturally incremental. Call graph: update edges for new/removed/changed call sites per commit. Module dependency: update when **import** statements change. Co-change graph: append new co-occurrence counts for files changed

together in the new commit. Ownership: update line-ownership fractions per commit (git blame delta). All are $O(\text{changed entities})$ updates — excellent for the KG-Commit incremental model.

Prior Implementations and Difficulty. Call graph: Soot/WALA (production-ready). Module dependency: JavaParser (trivial, <1 day). Co-change graph: custom git log mining (trivial, 1–2 days). Ownership: `git blame` wrapper (trivial). Difficulty: **very easy** to **easy** for all components.

Pros. Co-change graph captures implicit coupling not in the static dependency graph — files that always change together but have no import relationship. Ownership/expertise: low-ownership files have higher defect rates (proven in large-scale studies). Module dependency layering violations are structural defect predictors. All are naturally incremental — perfect for KG-Commit.

Cons. Co-change requires a historical baseline (cold-start problem for new repos). Context-sensitive call graph (SPARK/k-CFA) is expensive for large projects. Ownership metrics are noisy for files with many small contributors.

Java tooling. Soot/WALA (call graph), JavaParser (imports), git log (co-change, ownership).

17 Family 16: Binary and Compiled Representations

17.1 What It Captures

Java bytecode (opcode sequences, n -grams), binary CFG (reconstructed from disassembly), binary call graph, Bin2Vec (learned binary function embeddings), BinVuGAL (GNN over binary CFG), LLM-assisted decompilation.

Graph encoding.

```
BinaryFunction --BINARY_CFG_NEXT--> BinaryBlock
BinaryBlock --OPCODE_SEQ[pos]--> Opcode
BinaryFunction --BINARY_CALLS--> BinaryFunction
```

Efficiency. Binary CFGs are smaller than source CFGs (optimising compiler reduces branch count). Opcode n -grams: compact vectors. Binary function embeddings (Bin2Vec): 768-float vectors, same cost as LM embeddings. However, binary analysis tools (Ghidra, IDA Pro) are heavyweight and slow to initialise. Soot operating on `.class` files is efficient and avoids full binary disassembly.

Difference Modeling. Hard. Binary diffs are semantically opaque. Recompilation must happen per commit. Binary similarity tools are not designed for delta computation. For JVM bytecode via Soot: the `.class` files change whenever a `.java` file changes, so bytecode-level diff is tightly coupled to source-level diff. Mapping binary diff to source diff requires `LineNumberTable` metadata, which is present in debug builds.

Prior Implementations and Difficulty. Soot (bytecode analysis, production-ready). Ghidra (binary analysis, open-source, production-ready). Bin2Vec, BinVuGAL: research code. LLM-assisted decompilation (VulBinLLM 2025): very recent, not production-ready. Difficulty: Soot bytecode **easy**; binary-level analysis **hard**; decompilation **very**

hard.

Pros. Works without source code — covers vendored JARs and native JNI components. Language-agnostic across all JVM languages. Soot’s bytecode mode is the most reliable Java analysis option when source is unavailable.

Cons. Source is available for all Apache repos in the corpus — binary analysis adds complexity for no gain when source is accessible. Mapping binary locations back to source lines for diff alignment is fragile. Not needed for the primary use case.

Java tooling. Soot (bytecode), Ghidra, ASM, Bin2Vec (PyTorch).

18 Universal Graph Encoding of Non-Graph Representations

Any finite representation can be encoded as a graph. Table 2 gives the translation recipe.

Table 2: Encoding non-graph representations as KG sub-graphs.

Original form	Transformation	Graph model
Token sequence	Chain graph	Token -NEXT-> Token[pos]
n -gram bag	Shared-context star	Token -CONTEXT-> Token[+n]
Feature vector	Star graph	Entity -HAS_METRIC[name]-> Value
Coverage matrix	Bipartite graph	Test -COVERS-> Statement
Edit script	Event nodes	Commit -EDITS[action]-> ASTNode
Lattice value	Typed edge	Var -MAY_BE_NULL-> Deref
Path constraint	Node property	Block -PATH_COND-> Formula
Execution trace	Temporal chain	Event -NEXT[step]-> Event
Embedding vector	Node property	Node -EMBEDDING[model]-> Float[d]
Dynamic invariant	Typed edge	Method -INVARIANT-> Formula
Clone pair	Cross-link	Func -IS_CLONE_OF[type]-> Func
Review comment	Annotation node	DiffLine -HAS_REVIEW-> Comment

Table 2: Encoding non-graph representations as KG sub-graphs.

Original form	Transformation	Graph model
Stack trace	Frame chain	Frame[i] -NEXT_FRAME-> Frame[i+1]

19 Comprehensive Comparison

Family	Synt.	Ctrl.	Data	Cost	Δ diff	Leak risk
Token sequences	se- ✓	–	–	Very low	Trivial	None
AST	✓	–	–	Low	Easy	None
CFG	–	✓	–	Low	Moderate	None
CDG	–	✓	–	Low	Moderate	None
DFG / def-use	–	–	✓	Medium	Hard	None
PDG	–	✓	✓	Medium	Hard	None
CPG	✓	✓	✓	Medium	Very hard	None
Alias / heap	–	–	✓	High	V. hard	None
Taint paths	–	–	✓	Medium	Moderate	None
Abstract interp.	–	✓	✓	High	Moderate*	None
Symbolic exec.	–	✓	✓	V. high	Hard	None
Formal contracts	con- –	–	✓	Low	Easy	Possible
Program slicing	–	✓	✓	Medium	Moderate	None
IR / bytecode	✓	✓	✓	Medium	Hard	None
Exec. traces	–	✓	✓	High	N/A	High
Coverage matrices	–	–	✓	Medium	Moderate	High
AST edit scripts	✓	–	–	Low	Trivial	None
Delta CPG	✓	✓	✓	High	Moderate	None
Commit messages	mes- –	–	–	V. low	Trivial	None
Code comments	–	–	–	V. low	Trivial	None
Bug reports	–	–	–	Low	Trivial	Possible
Review comments	com- –	–	–	Low	Trivial	Possible
Code metrics	✓	✓	–	V. low	Trivial	None
Clone pairs	✓	–	✓	Medium	Moderate	None
Code smells	✓	✓	–	Low	Trivial	None
Learned embed.	✓	✓	✓	Medium	Easy	None
Co-change graph	–	–	–	Low	Trivial	None
Dev. ownership	–	–	–	V. low	Trivial	None

Family	Synt.	Ctrl.	Data	Cost	Δ diff	Leak risk
Binary / byte-code	✓	✓	–	Medium	Hard	None

* Infer reactive mode only.

20 Recommended Phased Integration Plan

1. **Phase 1 — Metrics + NL (Week 1–2).** CK metrics on changed functions. Commit message intent classification. SATD detection. All plug into existing `get_features()` with no new infrastructure.
2. **Phase 2 — AST + Edit Scripts.** Add AST subgraph (tree-sitter or Spoon). Run GumTree per commit to produce AST edit scripts. Store `EDITS[action]` edges on the Commit node. These are the most cost-effective semantic change edges available.
3. **Phase 3 — CFG + Call Graph (Week 4–6).** Add CFG edges and inter-procedural call graph (Soot). Derive cyclomatic complexity, # back-edges, reachable-method count. Connect call edges to existing FUNCTION nodes.
4. **Phase 4 — Data Flow / CPG.** Add def-use (DDG) edges via Soot Jimple, or generate full CPGs with Joern. At this point the structural backbone is complete.
5. **Phase 5 — Delta CPG.** Implement the before+after CPG merge script on top of Joern. Tag `CHANGED_IN(commit)`, `MAPPED_TO`, and `DELTA_STATUS` edges.
6. **Phase 6 — Typed Semantic Edges.** Run Infer (nullness/taint) in reactive mode per commit. Add `MAY_BE_NULL`, `TAINT_FLOWS_TO` edges. Run NullAway for annotation-based types.
7. **Phase 7 — Content Embeddings + GNN.** Initialise CPG nodes with GraphCodeBERT embeddings. Train an R-GCN or HGT over the change-annotated CPG. Compare against Phase 1–6 aggregate-vector baselines.
8. **Phase 8 — Deep Semantic Enrichment.** Add alias edges (WALA SPARK), exception flow graph (Robusta), concurrency lock graphs (RacerD) selectively for high-value components.

21 Open Research Questions

- **Semantic change-graph for JIT-DP.** Most SOTA (DeepJIT, CC2Vec, BiCC-BERT) encodes changes as flat sequences. A CPG-grounded delta graph for JIT-DP is largely unexplored.
- **Cross-scale message passing.** Fusing statement-level CPG subgraphs with project/author/issue/temporal KG nodes in one heterogeneous graph via message passing is an open design question.
- **Inter-procedural change effects.** Commit edits method A, defect surfaces in caller B. SDG-based propagation of change effects is rarely modelled at commit level.

- **Dynamic features without trace replay.** Can abstract interpretation approximate execution-trace features cheaply?
- **Review-comment-guided prediction.** Predicting which changed lines will receive critical review comments before review is a novel JIT framing.
- **Temporal decay of semantic features.** Time-aware GNNs combining temporal KG nodes with code subgraph features.
- **Exception-flow defect prediction.** Apache-project exception mishandling linked to ISSUE nodes mentioning specific exception types.

References

- [1] F. Yamaguchi et al., “Modeling and Discovering Vulnerabilities with Code Property Graphs,” *IEEE S&P*, 2014. <https://joern.io>
- [2] Y. Zhou et al., “Devign,” *NeurIPS*, 2019.
- [3] S. Wang et al., “GraphSPD,” *IEEE S&P*, 2023. https://csis.gmu.edu/ksun/publications/SP23_GraphSPD.pdf
- [4] Z. Ni et al., “BiCC-BERT,” *JSS*, 2024. <https://arxiv.org/pdf/2410.12107>
- [5] C. Cummins et al., “ProGraML,” *ICML*, 2021. <https://arxiv.org/abs/2003.10536>
- [6] J.-R. Falleri et al., “GumTree,” *ASE*, 2014; redesigned *ICSE*, 2024. <https://dl.acm.org/doi/10.1145/3597503.3639148>
- [7] B. Fluri et al., “ChangeDistiller,” *IEEE TSE*, 2007.
- [8] Z. Ni et al., “Heterogeneous Graph for Bug-Fixing Root Cause,” 2025. <https://arxiv.org/pdf/2505.01022>
- [9] Z. Ni et al., “JIT Defect Localisation via Code Graph,” *ICPC*, 2024.
- [10] D. Guo et al., “GraphCodeBERT,” *ICLR*, 2021.
- [11] T. Hoang et al., “CC2Vec,” *ICSE*, 2020.
- [12] H. Ni et al., “DeepJIT,” *MSR*, 2019.
- [13] Ferrante et al., “PDG,” *ACM TOPLAS*, 1987.
- [14] Horwitz et al., “SDG,” *ACM TOPLAS*, 1990.
- [15] Cousot & Cousot, “Abstract Interpretation,” *POPL*, 1977.
- [16] S. Chidamber, C. Kemerer, “CK Metrics,” *IEEE TSE*, 1994.
- [17] “Fault Localization with Code Coverage Representation Learning,” *ICSE*, 2021. <https://dl.acm.org/doi/abs/10.1109/ICSE43902.2021.00067>
- [18] “Combining Static and Dynamic Symbolic Traces,” *ICSE*, 2024. <https://dl.acm.org/doi/10.1145/3597503.3639212>
- [19] “Survey of Source Code Representations for ML Cybersecurity Tasks,” *ACM CSUR*, 2024. <https://arxiv.org/pdf/2403.10646>

- [20] “Deep Learning for Code Intelligence,” *ACM CSUR*, 2024. <https://arxiv.org/abs/2401.00288>
- [21] “TokDrift,” 2024. <https://arxiv.org/pdf/2510.14972>
- [22] S. Dasgupta et al., “IR2Vec,” *ACM TACO*, 2020. <https://dl.acm.org/doi/fullHtml/10.1145/3418463>